Exploitation de la mémoire virtuelle

Clément Ballabriga, Gilles Grimaud, Giuseppe Lipari

Mars 2026

Contents

1 Avant Propos **4**

2 Mémoire Virtuelle x86 **4**
2.1 Principe Général 4
2.2 Fonctionnement 5
2.3 Structures de données 8

3 Implémentation **10**
Q1 *Page Directory Entries* 10
Q2 *Page Table Entries* 10
Q3 un *page directory* 10
Q4 une *page table* 10

4 Premier usage **11**
4.1 Plan d'adressage 11
Q5 adresses de compilation 12
Q6 mémoire virtuelle noyau 12
Q7 espace mémoire 12
Q8 plan d'adressage 12
Q9 `setup_mmu()` 13
Q10 mise en oeuvre 13
4.2 Un programme *user land* 13
Q11 espace *user* 13
Q12 plan d'adressage *user* 14
Q13 *handler* de fautes 14
Q14 programme *user* 14
Q15 édition de lien *user* 14
Q16 activation *user land* 15
4.3 Correction 15
Q17 *Page Fault* v2.0 16
Q18 chercher la faute 16
4.4 Interposition 16
Q19 handler *sys call* 17
Q20 *sys call* utilisateur 17
Q21 *sys call* v2.0 17
Q22 Interposition d'appel 18

5 *Stack guard* 18
5.1 protection de pile . . . 18
Q23 dimensionnement du *user space* . . . 18
Q24 pages et adresses . . . 19
Q25 nouveau `setup_mmu` . . . 19
Q26 débordement de pile . . . 19
Q27 test de débordement . . . 19
Q28 message “stack overflow” . . . 19
Q29 analyse . . . 20
5.2 Extension dynamique de pile . . . 20
Q30 Structure d’allocation . . . 20
Q31 initialisation . . . 21
Q32 `void *alloc_page()` . . . 21
Q33 `void free_page(void *)` . . . 21
Q34 Pile d’appel virtuelle . . . 21
Q35 Implémentation . . . 21

6 Processus lourd 22
6.1 Copie mémoire . . . 23
Q36 `void page_copy(...)` . . . 23
Q37 `clone_page_directory(...)` . . . 24
Q38 `free_page_directory(...)` . . . 24
6.2 Activation . . . 24
Q39 *kernel stack* . . . 24
Q40 Contextes d’exécution . . . 25
Q41 allocateur de `struct ctx_s` . . . 25
Q42 Un premier `ctx_s` . . . 26
6.3 Changement de processus . . . 26
Q43 *syscall* `getpid` . . . 26
Q44 `switch_to` v1.0 . . . 26
Q45 *syscall* `yield` . . . 27
6.4 clonage à chaud . . . 27
Q46 `switch_to` v2.0 . . . 27
Q47 *syscall* `fork()` . . . 27
Q48 retour de `fork()` . . . 27
6.5 Terminaison . . . 28
Q49 *syscall* `exit` . . . 28
Q50 user `_entry_point` . . . 28

6.6 Validation 28
Q51 Sécurité 28
6.7 Clonage virtuel 28

7 . . . To Be Continued . . . 28

1 Avant Propos

En cours vous avez vu qu'un système d'exploitation sert fondamentalement trois intentions :

1. gérer le matériel pour en optimiser l'utilisation ;
2. fournir des abstractions pour les applications ;
3. assurer la sécurité du dispositif informatique, que ce soit pour isoler les défaillances ou pour protéger les logiciels de comportements malveillants.

Dans ce TP vous allez vous intéresser à la configuration d'un dispositif matériel destiné à faciliter la gestion de la mémoire (point 1) mais aussi à en assurer l'isolation et la protection (point 3). Ce dispositif, présent sur de nombreuses familles de microprocesseurs, est appelé MMU pour *Memory Management Unit*.

À l'aide de ce matériel vous allez illustrer deux paradigmes essentiels de la programmation des systèmes :

L'interposition : qui consiste à imposer l'exécution d'un traitement système tier en préambule à l'exécution du service réellement sollicité par une application ;

La virtualisation : qui consiste à donner l'illusion de l'utilisation d'un matériel « qui pourrait être, mais qui n'est pas ».

Vous expérimenterez l'utilisation d'une MMU sur les microprocesseurs de la famille x86 en mode protégé 32 bits. Il n'y a pas de différence fondamentale avec le mode 64 bits ; simplement, les adresses étant sur 64 bits, il y a plus de tables d'indirection à configurer, ce qui alourdit inutilement le temps dévolu aux travaux pratiques.

Votre enseignant de TD va vous demander de « forker » un projet `my-kernel`. Chaque question de ce sujet consiste soit :

1. en la rédaction d'une réponse à la question formulée dans un fichier `rendu.md` à la racine du projet, soit ;
2. en ajoutant le code dans le dépot que vous aurez « forké ».

N.B. : Chaque modification doit être suivie d'un `commit` avec un message clair !

2 Mémoire Virtuelle x86

2.1 Principe Général

La MMU (*Memory Management Unit*) est une composante des architectures Intel x86. Elle distingue 2 états de fonctionnement du microprocesseur :

- **L'état « *user* » :** est l'état dans lequel est le microprocesseur lorsqu'il exécute une application (même si l'utilisateur de cette application est le *root* d'un système Unix par exemple).
- **L'état « *kernel* » (aussi appelé *supervisor*) :** est l'état dans lequel est le microprocesseur :

  1. au démarrage de la machine (en pratique les choses sont plus compliquées sur les processeurs intel) et ;
  2. lorsqu'il exécute le code associé à une interruption matérielle, ou logicielle.

En mode *kernel*, les registres de la MMU peuvent être reconfigurés ; en mode *user*, cela produit une faute (une sorte d'interruption logicielle). En fait, en mode *user*, les instructions et les registres associées à des éléments de configuration du matériel sont interdites et produisent des fautes. Seul le *kernel* peut gérer le matériel et donc exécuter ces opérations, cela fait précisément partie de ses fonctions…

Les microprocesseurs de la famille intel définissent (principalement) 3 registres, `CR0`, `CR2` et `CR3` pour configurer la MMU en mode protégé 32 bits (mais aussi 64 bits).

Le registre `CR0` (CR signifie *Control Registers*) définit un certain nombre de bits qui indiquent si tel ou tel mécanisme du microprocesseur doit être mis en oeuvre ou « débrayé ». En l'occurence, le bit 31 indique si le dispositif MMU est activé (le bit vaut 1) ou s'il est désactivé (le bit vaut 0). Au démarrage la MMU, qui n'est pas encore configurée, n'est pas activée non plus (le bit 31 de CR0 est à 0). Pour activer la MMU (Intel parle de « Paging » d'où le nom donné au bit 31 : `PG`), il faut que le programme (le système d'exploitation) fasse passer le bit `PG` de `CR0` à 1. Cela doit être fait après que le dispositif MMU ait été correctement initialisé, faute de quoi l'activation résultera en un comportement indéterminé de la MMU. Le second bit utile à la MMU dans le registre `CR0` est le bit 16. Lorsque ce bit est positionné à 1, il indique que même en mode *kernel*, la MMU doit générer une faute lorsque le code demande à écrire une adresse mémoire configurée « *Read-Only* », comme il le ferait en mode *user*. Si le bit est à 0, alors le code (*kernel*) pourra écrire à une adresse mémoire configurée *Read-Only* pour le code *user* (qui, lui, ne pourra pas écrire quoi qu'il en soit).

Par ailleurs, lorsqu'une faute est produite par la MMU, le registre `CR2` contient l'adresse que l'instruction fautive a tenté d'accéder.

Le registre `CR3` donne l'adresse, en mémoire physique, d'une table appelée *Page Table Directory*. Les 12 bits de poids faible de cette adresse doivent être nuls (positionnés à zéro). Une *page* de mémoire est un bloc de 4096 (4 Ko) adresses consecutifs en mémoire. Cela signifie que l'adresse d'une *Page Table Directory* est nécessairement l'adresse de début d'une page de mémoire.

2.2 Fonctionnement

Pour mémoire, en mode protégé 32 bits, lorsqu'elle est activée, la MMU des processeurs intel fonctionne comme suit.

Dans sa configuration la plus courante, la MMU représente la mémoire comme un ensemble de pages mémoires de 4096 octets (4Ko). La première page (la page 0) de mémoire physique correspond aux octets dont les adresses vont de 0 à 4095, la seconde page (la page 1) de mémoire physique commence à l'adresse 4096 et fini à l'adresse 8191, la troisième commence à 8192 et fini à 12287, etc. Ainsi toutes les pages commencent à une adresse multiple de 4096, c'est-à-dire, à une adresse dont les 12 bits de poids faible sont positionnés à 0. Les 20 bits de poids forts donne le numéro de la page. De plus, toute adresse mémoire correspond à l'adresse d'un octet dans une page, pour retrouver la page, il suffit de consulter les 20 bits de poids fort de l'adresse. Les 12 bits de poids faible sont appelés offset de page.

Lorsque le microprocesseur demande à accéder à une adresse en mémoire, cette adresse, qui est un nombre codé sur 32 bits (mode protégé 32 bits), est alors dite virtuelle. Le dispositif de MMUs exécute ainsi les étapes suivantes :

Découpage de l'adresse virtuelle : Les 10 bits de poids fort de cette adresse virtuelle sont appelés `idx1` et les 10 bits suivants sont appelés `idx2`. Enfin, comme cela a déjà été dit, les 12 derniers bits (de poids faible) sont appelés `offset` de page.

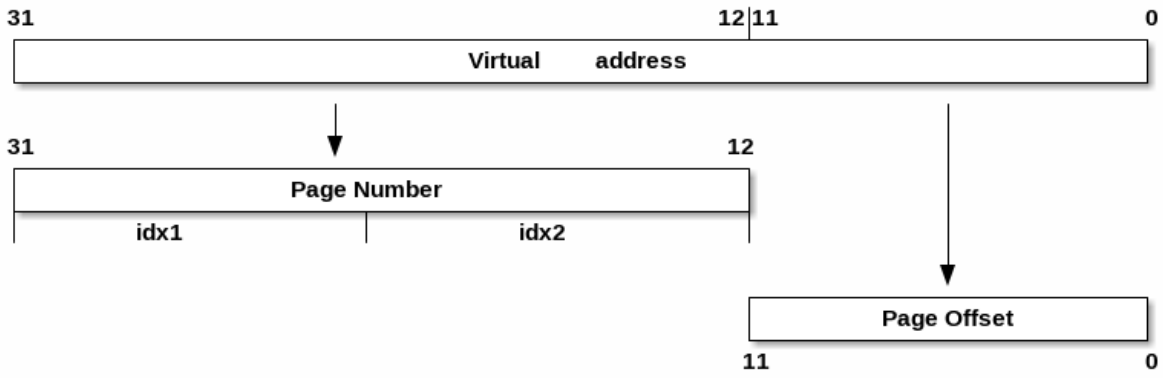


Figure 1: Decoupage de l'adresse virtuelle

Determination de la page table associée à idx1 : Comme indiqué précédemment, le registre `CR3` donne l'adresse d'une table, appelée *Page Table Directory*, de 1024 entrées. La MMU consulte cette table à l'index `idx1`. L'entrée, sur 32 bits, code différentes informations, dont :

1. l'indication selon laquelle cet index est valide ou pas, s'il ne l'est pas, la MMU produit une faute ;
2. l'indication selon laquelle cet index est réservé au mode kernel ou pas, s'il l'est et que le code exécuté est en mode user, la MMU produit une faute ;
3. les 20 bits de poids fort de l'adresse d'une table, appelée *Page Table*, de 1024 entrées. Les 12 bits de poids faible de l'adresse de début de cette table sont forcement à 0, et ne figurent pas dans la *Page Table Directory*.

Détermination de la page physique associée à idx2 : la MMU consulte l'entrée idx2 de la *page table* déterminée par l'étape 2. L'entrée, sur 32 bits, code différentes informations, dont :

1. l'indication selon laquelle cet index est valide ou pas, s'il ne l'est pas, la MMU produit une faute ;
2. l'indication selon laquelle cet index est réservé au mode kernel ou pas, s'il l'est et que le code exécuté est en mode *user*, la MMU produit une faute ;
3. les 20 bits de poids fort de l'adresse d'une « page de mémoire physique » associée à `(idx1, idx2)`.

Accès à l'adresse physique : La MMU utilise alors l'adresse de la page physique déterminée par l'étape 3. Cette adresse a la particularité, d'être l'adresse d'une page, et donc, d'avoir les 12 bits de poids faibles positionnés à 0. Ces 12 bits sont remplacés par les 12 bits d'offset de page, produits par l'étape 1. L'adresse complète ainsi formée est l'adresse physique qui est finalement accédé.

Grâce à ce dispositif de MMU, il devient possible d'associer à chaque page de mémoire virtuelle, une page de mémoire physique quelconque, ou de s'assurer qu'en tentant d'accéder à cette page, une application produira une faute que le système d'exploitation pourra récupérer sous la forme d'une interruption.

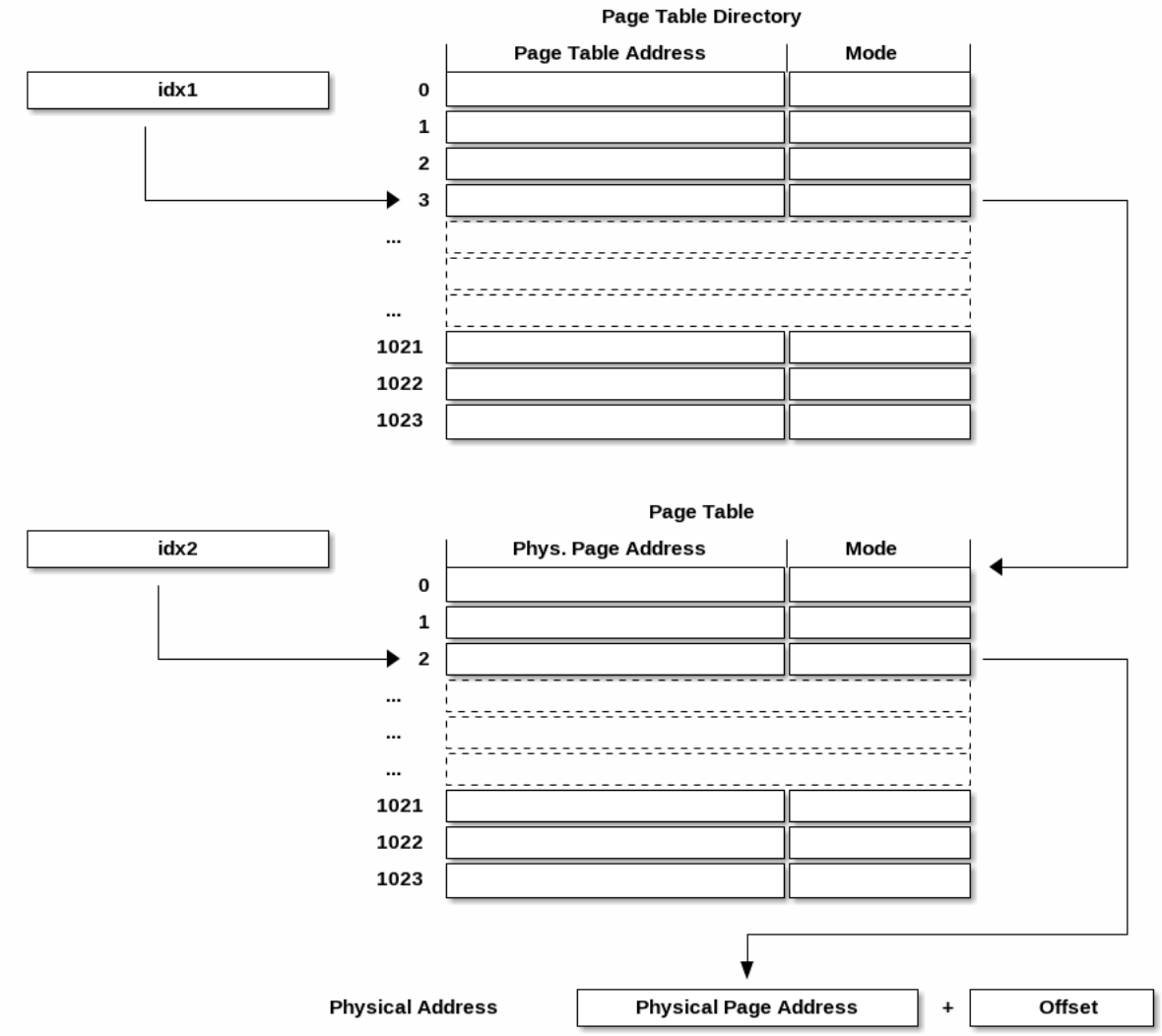


Figure 2: Traduction d'un adresse virtuelle en adresse physique

2.3 Structures de données

Dans le détail, chaque entrée d'une *Page Table Directory* est un champ de 32 bits organisé comme suit :

bit(s)	Description
31-12	Bits 31-12 de la page physique associée à cette entrée. Cette page physique correspond à une *Page Table*.
11-8	0- Non utilisés.

7	0- Taille d'une page mémoire (0=> 4Ko, 1=> 4Mo)
6	0- Non utilisé.
5	0- Entrée utilisée : la MMU met ce bit à 1 lorsqu'elle l'utilise.
4	0- Cache désactivé (les données ne sont pas copiées en cache lorsque ce bit est à 1).
3	0- Gestion du cache (0=> write-through, 1=> write back).
2	Entrée autorisée en mode *User* (1) ou *kernel* (0). L'accès à une entrée *kernel* en mode *user* produit une faute.
1	Entrée autorisée en lecture/écriture (1) ou lecture seulement (0). Si ce bit est 0, l'accès en écriture produit une faute.
0	L'entrée est "présente" (1) ou invalide (0). L'accès à une entrée invalide produit une faute.

Dans le détail, chaque entrée d'une *Page Table* est un champ de 32 bits organisé comme suit :

bit(s)	**Description** ——————————————————————-: 31-12	Bits 31-12 de la page
	ée à cette entrée.	Cette page physique est la page associée à la page	virtuelle.
11-9	0- Non utilisés.
8	0- Sinon l'entrée et déclarée « globale ». Elle doit être la même dans toutes les tables de MMU, et elle n'est pas invalidée lors d'un changement de table…
7	0- (gestion des caches non utilisée dans ce sujet).
6	0- « *dirty bit* » (la MMU le met à 1 lorsque la page est accédée en écriture (une donnée est modifiée).
5	0- La MMU met ce bit à 1 lorsque la page est accédée en lecture.
4	0- Cache désactivé (les données ne sont pas copiées en cache lorsque ce bit est à 1).
3	0- Gestion du cache (0=> write-through, 1=> write back).
2	Entrée autorisée en mode *User* (1) ou *kernel* (0). L'accès à une entrée *kernel* en mode *user* produit une faute.
1	Entrée autorisée en lecture/écriture (1) ou lecture seulement (0). Si ce bit est à 0, l'accès en écriture produit une faute.

0	L'entrée est "présente" (1) ou invalide (0). L'accès à une entrée invalide produit une faute.

3 Implémentation

Vous allez définir des structures de données nécessaires pour déclarer, en C un tableau `page_directory_entry` de 1024 `struct Directory_Entry_s`. Chaque entrée sera de 32 bits. Pour cela vous pourrez utiliser la notation en *bit field* du langage C.

Q1 *Page Directory Entries*

Créez un fichier `mmu.c` dans lequel vous déclarerez une structure `struct Directory_Entry_s` .

Q2 *Page Table Entries*

Dans le fichier `mmu.c` déclarez une structure `struct Page_Entry_s` correspondant à une entrée d'une table *Page Table*.

Q3 un *page directory*

Dans le fichier `mmu.c` déclarez une table globale `page_directory` composée de 1024 structures `Directory_Entry_s`. Vous utiliserez l'attribut d'alignement pour vous assurer que le compilateur choisisse une adresse alignée sur une page de mémoire physique (donc alignée sur un multiple de 4096, vous pourrez définir une macro PAGE_SIZE).

Avec `gcc` l'attribut d'alignement s'écrit comme suit :

```
1  int t  __attribute__((aligned (16)));
```

Ici l'adresse de la variable t est un multiple de 16.

Q4 une *page table*

Dans le fichier `mmu.c` déclarez une table globale `page_entry` composée de 1024 structures `Page_Entry_s`. Vous utiliserez l'attribut d'alignement pour vous assurer que le compilateur choisisse une adresse alignée sur une page de mémoire physique (donc alignée sur un multiple de PAGE_SIZE).

4 Premier usage

Dans un premier temps, nous allons étudier la possibilité d'utiliser la MMU pour isoler les bugs et les comportements malveillants d'un « programme utilisateur » du système d'exploitation. En effet, lorsqu'un programme ne fonctionne pas correctement, il est possible qu'il modifie les structures de données et/ou le code du système d'exploitation lui-même. Dans ce cas, le plantage (ou le comportement malveillant) d'une application peut entraîner le plantage (ou la compromission) du système d'exploitation lui-même. Pour éviter cela, le système d'exploitation « s'isole » des applications, en configurant la MMU de tel sorte que l'application (qui sera exécutée en mode *user*) ne puisse pas modifier la mémoire utilisée par le système d'exploitation lui-même.

Pour cela nous allons reprendre le code de base de `my-kernel` (dépôt git donné par l'enseignant). Le but est de le faire évoluer pour pouvoir activiter la MMU et exécuter le code contenu dans un module `user.o`. Ce code sera exécuté en mode *user* du microprocesseur, avec la MMU activée de telle sorte que le module `user.o` n'ait pas accès à la mémoire du reste de `my-kernel`.

4.1 Plan d'adressage

Pour pouvoir isoler le système d'exploitation du programme de l'utilisateur, il faut définir un plan d'adressage. Un plan d'adressage indique, pour chaque élément logiciel la place (les adresses) qu'il occupe en mémoire. Comme vous le savez, un programme exécutable est composé de plusieurs sections :

La `section .text`: cette section du fichier binaire contient le code machine à exécuter.

Les `section .data` et `section .bss`: ces sections contiennent les données globales du programme. La `section .data` contient les données initialisées alors que la `section .bss` contient les données non initialisées. Dans la section data certains compilateurs distinguent la `section .rodata` qui ne peut que être lue, et la section `.data` qui contient des données qui peuvent être lues et modifiées.

La `section .stack` cette section contient la pile d'appel, avec les variables locales.

Par défaut, le compilateur choisit des adresses en conformité avec le système d'exploitation qui va lancer le programme. Cependant, nous ne compilons pas ici un logiciel lancé par un système d'exploitation, mais directement « booté » par le logiciel d'amorce `grub`. Ce logiciel charge le binaire à exécuter, en mémoire physique, à une adresse fixe.

Dans le `Makefile` le linkeur (qui choisit la place en mémoire de chaque élement du binaire une fois compilée) est appelé avec une option peu commune. Il s'agit de l'option `-T`, qui configure le compilateur avec un « *link script* ». Ce fichier précise au linkeur à quelle adresse doit être placé quel élement du code compilé.

Q5 adresses de compilation

Selon le « *link script* » désigné par le `Makefile`, à quelle adresse est placée le code compilé, en mémoire ? Où sont placées les autres sections du programme compilé ?

Q6 mémoire virtuelle noyau

Au moment où la mémoire virtuelle est activée, l'accès à la mémoire physique pour lire la prochaine instruction à exécuter est remplacée par un accès à la mémoire virtuelle. Pour que ce changement d'adresse que la virtualisation de la mémoire impose soudainement au microprocesseur se passe au mieux, le plus simple est que l'adresse virtuelle corresponde à ce moment là, à l'adresse physique.

De plus, pour son affichage, votre programme utilise la fonction `putc` qui écrit dans la mémoire de la carte vidéo pour afficher les caractères à l'écran. Vous pouvez lire dans `src/main.c` :

```
1 /* base address for the video output assume to be set as character
     oriented by the multiboot */
2 unsigned char *video_memory = (unsigned char *) 0xB8000;
```

Calculez les valeurs de idx1 et de idx2 de l'adresse à laquelle le code de `my-kernel` est chargé en mémoire.

Puis calculez les valeurs de idx1 et de idx2 de l'adresse de la mémoire vidéo utilisée par la fonction `putc`.

Q7 espace mémoire

Vous allez définir une configuration de mémoire virtuelle minimale fonctionnelle. On suppose que vous ne configurez que l'entrée idx1 du *Page Table Directory* vers une *Page Table*, et toutes les entrées de cette *Page Table* (dont idx2).

Combien de pages mémoires seront finalement configurées ? ce qui correspond à combien d'octets de mémoire ?

Q8 plan d'adressage

Le but de cette configuration minimale est de s'assurer que les adresses virtuelles du code de `my-kernel` correspondent aux adresses physiques. Ce faisant, l'activation de la MMU ne mettra pas en péril l'exécution du binaire de `my-kernel`.

Proposez, de manière informelle, une configuration minimale des tables de MMU pour que les adresses physiques du binaire de `my-kernel` soient les mêmes que les adresses virtuelles configurées par la MMU.

Q9 `setup_mmu()`

Implémentez, dans le fichier `mmu.c` une fonction `void setup_mmu()`. Cette fonction doit :

1. initialiser toutes les entrées de la table `page_directory` pour qu'elles soient invalides (non présentes) ;
2. initialiser toutes les entrées de la table `page_table` pour qu'elle associe à chaque entrée virtuelle la même page en adresse physique (*e.g.* la page virtuelle 256 sera associée à la page physique 256) ;
3. initialiser l'entrée `idx1` avec l'adresse de la table `page_directory` pour qu'elle fasse référence à la `page_table` ;
4. initialiser les registres `CR3` et finalement `CR0` pour activer la MMU.

Q10 mise en oeuvre

Enfin, dans la fonction `main` de `my-kernel`, appelez votre fonction `setup_mmu()` après que les autres « setup » soient exécutés. Puis vérifiez, qu'avec la MMU activée, votre noyau continue de fonctionner.

4.2 Un programme *user land*

Vous avez initialisé la MMU pour qu'elle fasse un mapping identité entre les adresses physiques du noyau, et les adresses virtuelles. Ce mapping associe les 4 premiers mega-octets de la mémoire physique au 4 premiers mega-octets de la mémoire virtuelle. Cependant, parce que le bit *uk* des `Page_Entry` est à 1 seul un code exécuté en mode *kernel* peut s'exécuter dans cet espace d'adressage virtuel.

Vous allez maintenant créer un second espace d'adresse, exécutable par le microprocesseur en mode *user*. Ce second espace d'adressage virtuel sera destiné à héberger un « programme » qui n'aura pas les privilèges du noyau. Il sera dans la même situation de droit CPU que les programmes que vous exécutez habituellement sur votre ordinateur. On parle de programme « *user land* », ce monde magique où les plantages de vos codes n'entraînent pas un plantage de l'ordinateur…

Pour cela, la première étape consiste à définir un nouvel ensemble d'adresses virtuelles, qui seront mappées avec la fonction identité à des adresses physiques, mais avec des droits *user* et *read/write*.

Q11 espace *user*

Modifiez, dans le fichier `mmu.c`, la fonction `void setup_mmu()`, afin que les 4 mega-octets de mémoire qui suivent ceux dédiés au *kernel* soient accessibles en lecture et écriture, pour un programme

en mode utilisateur. Notez que 4 Mos constituent une nouvelle entrée de `Page_Directory` et 1024 entrées de `Page_Table`. Cette nouvelle table sera appelée `user_page_table`. Pensez à la déclarer.

Q12 plan d'adressage *user*

Déterminez quelle plage d'adresses (virtuelles) vous avez configuré en mode *user*. Quelle est la plus petite adresse accessible en mode *user* selon votre configuration ? Et quelle est la plus grande adresse accessible ? Indiquez votre réponse dans le fichier `rendu.md`.

Q13 *handler* de fautes

Lorsque le programme utilisateur effectura un accès illégal à la mémoire, par exemple en essayant de lire ou d'écrire dans la mémoire noyau, cela produira une faute MMU. Cette faute est une forme d'interruption. Le niveau d'interruption associé aux fautes de page de la MMU est 14. Le fichier d'interface du vecteur d'interruption `idt.h` définit une fonction spécifique pour enregistrer dans l'IDT les handlers des « interruptions logicielles » (c'est-à-dire, des interruptions qui viennent d'un logiciel et non d'un matériel). On lit dans `include/idt.h` :

```
1  void idt_setup_int_handler(int intno, void (*handler)(int_regs_t *r));
```

Modifiez votre fonction `main()` pour qu'elle configure l'idt de telle sorte qu'en cas de faute de page, la fonction `page_fault_handler()` soit appelée (un peu comme les `empty_handler` le sont). Pour l'instant la fonction `page_fault_handler()` se contentera d'afficher un message d'erreur.

Q14 programme *user*

Il faut maintenant compiler un « programme *user land* » qui s'exécutera à cette adresse mémoire. Comme le `Makefile` fournit déjà un moyen de produire un code binaire bootable, il vous suffit de réutiliser cette chaîne de compilation, pour qu'elle compile, en plus du noyau, un fichier c `src/user.c` qui déclare une fonction `void user_main()`. Cette fonction pourra simplement appeler `puts("Hello world from user land!\n")` puis faire une boucle infinie pour fixer le CPU en *user land*.

Q15 édition de lien *user*

Cependant, le problème, c'est qu'une fois compilé, le fichier user.o est linké avec le reste du code de my-kernel. Il ne sera donc pas placé par le linkeur à l'adresse que vous avez configuré en mode *user*.

Pour que spécifiquement le code contenu dans ce fichier débute à l'adresse que vous avez calculé à la question Q12, il faut modifier le *link script* utilisé pour l'édition de lien.

Pour cela, il vous suffit de déclarer une nouvelle section, par exemple `user_land` suivie de l'adresse à laquelle vous souhaitez que cette section commence, comme le fait la section `.text` par exemple.

Au sein des accolades vous déclarez simplement la liste des fichiers `.o` à mettre dans cette section, un fichier par ligne. Et pour chacun d'eux, entre parenthèses, la liste des sections (.text, .data, .rodata, .bss) du `.o` que vous voulez placer dans la section `user_land` de l'image binaire finale.

Une fois la configuration du *link script* terminée, vérifiez que le makefile produit bien un binaire au sein duquel la fonction `user_main` est au sein de l'espace d'adressage virtuel *user*.

Q16 activation *user land*

Il s'agit enfin d'exécuter la fonction `user_main` du module `src\user.o` à la fin de votre séquence de boot (à la fin du main de `my-kernel`). Vous pourriez bien sûr appeler la fonction `user_main`, mais cet appel de fonction ne changerait pas l'état de fonctionnement du CPU qui est en mode *kernel*, et donc, les restrictions d'accès à la mémoire du *user land* ne seraient pas faites, par la MMU.

Pour faire cet appel il faut quitter le mode *kernel* et pour cela, le plus simple est d'utiliser l'instruction assembleur `iret` qui « retourne depuis une interruption » ; de telle sorte que le retour, en mode *user* débute l'exécution de la fonction cible. Comme la construction d'une pile de retour exploitable par l'instruction `iret` est délicate (pleine de détails techniques), le code de base de my-kernel vous en fournit une implémentation dans `src/call_user.s`. Le fichier `include/call_user.h` donne le prototype de la fonction `call_user`.

Modifiez votre fonction `main` pour que la séquence de boot se termine par un appel en *user land* à la fonction `user_main`, en indiquant, comme pile, une pile de 64Ko déclarée dans le même fichier que `user_main`.

Indiquez dans `rendu.md` ce que vous observez, et pourquoi cela se passe ainsi.

4.3 Correction

Il s'agit donc maintenant de corriger le comportement incorrect précédent. A ce stade, lorsque le programme *user land* faute, le comportement attendu est d'afficher un message d'erreur, indiquant l'adresse mémoire qui a echoué, et l'adresse de l'instruction qui a produit cette faute, puis de geler le système, pour que l'utilisateur puisse lire l'erreur et éteindre le système en échec.

Q17 *Page Fault* v2.0

Proposez une nouvelle implémentation du handler de *page fault* afin qu'il se comporte comme expliqué précédemment. L'adresse de la tentative d'accés peut être lue dans le registre de controle adéquat. L'adresse de l'instruction qui a fauté peut être retrouvée dans la structure de donnée passée en paramêtre du handler. Cette structure donne l'état du microprocesseur au moment de l'interruption, et donc, notamment, la valeur du registre `%eip` (i.e. *instruction pointer* sur intel x86 32Bits).

Q18 chercher la faute

Indiquez les informations que vous lisez maintenant lorsque l'erreur se produit, et expliquer (en précisant les commandes shell que vous utilisez) comment vous pouvez trouver la raison de l'erreur avec ces informations. Pour mémoire une fois compilé, le binaire exécutable est écrit dans le fichier au format .elf `iso/boot/mykernel.bin`.

4.4 Interposition

Le problème, identifié dans la section présente, vient du fait qu'il faut pouvoir appeler des fonctions systèmes (on parle parfois de services) depuis l'espace *user*. Le plus simple serait de donner le droit à l'utilisateur d'accéder à la mémoire du système, mais c'est précisement ce que l'on veut empêcher pour qu'en cas de bug du programme utilisateur le programme système puisse continuer de fonctionner. De fait, lorsqu'une interruption (ou une faute) se déclanche, le programme utilisateur est bien interrompu, et le programme *kernel* exécute une fonction : le handler d'interruption. Les microprocesseurs prévoient le même procédé pour transférer, à la demande de l'application, le flot d'exécution depuis le mode *user* vers le mode *kernel*, à travers un mécanisme « d'interruption logicielle ».

Pour permettre aux programmes utilisateurs d'appeler des fonctions systèmes (on parle de *sys call*), les applications vont donc déclancher des interruptions logicielles. Sur les processeurs x86 il est possible pour un programme exécuté en mode *user* de déclancher une telle interruption en exécutant simplement l'instruction assembleur :

```
1    int $87
```

ou 87 est un numero d'interruption sollicité. Lorsque le microprocesseur exécute cette instruction il se comporte comme si un matériel sollicité une interruption (ici la numero 87). Cela déclanche donc l'appel, en mode *kernel* d'un handler d'interruption attaché à l'interruption 87.

Pour « capturer » cette interruption dans le noyau, il faut enregistrer un handler d'interruption, comme cela a été fait jusqu'à présent, avec la fonction `void idt_setup_int_handler(int,void (* handler)(int_regs_t *r));`

N.B. En pratique, sur les architectures x86, il existe d'autres instructions microprocesseurs que `int` qui peuvent être utilisées pour implémenter un *sys call*. Il s'agit notamment de l'instruction `callf` ou encore et de l'instruction `sysenter` (bien que cette dernière n'est pas le même sens sur les processeurs intel et les processeur AMD). Nous traitons ici avec l'instruction `int` pour éviter d'avoir à introduire des notions spécifiques aux microprocesseurs de la famille x86 (notion de segmentation principalement).

Q19 handler *sys call*

Proposez une modification de votre programme principal pour qu'il traite l'interruption logicielle 87. Ce handler d'interruption est appelé lorsque un programme utilisateur sollicite une fonction système. Pour l'instant, la fonction système que nous souhaitons pouvoir appelée est `puts`. le paramêtre de cette fonction est l'adresse de la chaine de caractère à afficher. Il va falloir passer cet argument au handler d'interruptions logicielles. Puisque le handler recoit l'état des registres du programme interrompu, le plus simple est de mettre ce paramêtre dans un registre. Vous pouvez (avantagueusement) supposer que le registre `%edx` est utilisé par le programme *user land* à cette fin.

Q20 *sys call* utilisateur

Il faut maintenant modifier le programme `src/user.c` pour que **sa** fonction `puts` consiste à déclancher l'interruption logicielle 87 après avoir écrit le paramêtre dans le registre `%edx`.

Comme le noyau déclare déjà une fonction `puts` et que vous allez en déclarer une autre dans le même binaire, il va y avoir un conflit de nommage au moment de la compilation. renommez `puts` en `kputs` dans `src/main.c` et déclarez votre nouvelle fonction `puts` dans le fichier `src/user.c`. Profitez-en pour retirer la déclaration de la fonction `extern puts(char *s);` de ce fichier.

Q21 *sys call* v2.0

Pour l'instant, l'interruption logicielle 87 ne permet de soliciter qu'un seul service, la fonction `puts`. Cependant nous voulons aussi pouvoir exposer d'autres fonctions. Il s'agit, dans un premier temps, de : `putc` et `clear_screen`. D'autre pourront suivre.

Nous pourrions déclarer un handler pour chaque fonction, mais comme le nombre de fonction système est suspetible d'augmenter avec le futur, alors que le nombre d'interruption est limité à 256, nous allons utiliser un autre moyen. Lors de l'exécution d'une interruption logicielle le registre `%eax` contiendra un nombre 32 bits qui indiquera le numero du service système demandé par le programme utilisateur. Implémentez une nouvelle fonction pour `putc` et `clear_screen` dans `src/user.c`, puis modifiez `sys_call_handler` dans `src/main.c` en conséquence. Testez vos nouvelles fonctions.

Q22 Interposition d'appel

Expliquez en quoi cette mise en oeuvre des *sys call* (celle que vous avez faite) protège le noyau des comportements d'un programme utilisateur malveillant ?

5 *Stack guard*

Dans cette section vous allez mettre en place un système de virtualisation de la mémoire accéssible à l'utilisateur afin de gérer au mieux la pile d'exécution. Pour l'instant la pile d'exécution est simplement un tableau global déclaré par le programme `src/user.c` et utilisé comme pile d'appel au moment de l'activation de l'espace d'adressage utilisateur (via l'appel a `call_user`).

Cette manière de faire n'est pas satisfaisante car :

1. si la pile d'exécution déborde, aucune erreur ne sera produite, les variables locales vont simplement écraser les autres informations placées là par le compilateur ;
2. la pile d'exécution est entièrement réservée *a priori* et consomme donc potentiellement plus de mémoire que nécessaire, ce qui prive inutilement les autres programmes de cette ressource.

Vous allez traiter succéssivement ces deux points.

5.1 protection de pile

Tout dabord, vous pouvez noter que la configuration actuelle de la MMU associe toutes les pages de la deuxième entrée du `page_directory` à une page de mémoire physique. Cela n'est pas utile ici, puisque votre programme est bien plus petit que cela. Pour allouer seulement la mémoire utile il faut connaitre la dernière adresse du module `user.o`. Pour cela, il est possible de déclarer, dans la section de votre choix du *linkscript* des étiquettes :

```
1 label=.;
```

Ce faisant le linkeur considèrera qu'il existe une variable **`label`** à l'adresse ou l'étiquette est lue. En demandant l'adresse de cette variable, il devient possible de connaitre l'adresse de n'importe-quelle partie de ce que le compilateur a généré.

Q23 dimensionnement du *user space*

Modifiez le *linkscript* pour déclarer des variables `_begin_of_user_space` et `_end_of_user_space`, qui seront respectivement au début et à la fin de la mémoire (et donc de la section) utilisée par le module `user.o`.

Q24 pages et adresses

Implementez une fonction utilitaire `void *page_to_addr(int page)` qui retourne l'adresse, sous la forme d'un `void *` pour la page donnée en argument, ainsi que sa réciproque : `int addr_to_page(void *addr)`.

Q25 nouveau `setup_mmu`

Modifiez la fonction `setup_mmu` pour qu'elle n'associe que les pages de mémoire physique utiles à votre module `user.o` dans le `page_directory` utilisé par la mmu. Verifiez que votre noyau fonctionne toujours.

Q26 débordement de pile

Pour eviter les débordements de pile, vous allez simplement modifier la configuration de la mmu de tel sorte que toute tentative d'accés à la dernière page de la pile produise une faute. De cette manière, lorsque le programme *user* aura empilé trop d'appel de fonction (par exemple dans une fonction récursive) son programme finira par écrire sur cette page protégée et cela produira une erreur que le noyau pourra intercepté, plutot que de laisser le programme continuer à s'exécuter en écrasant ses propres variables globales et/ou fonctions.

Q27 test de débordement

Modifiez le programme `user.c` pour tester qu'un débordement de pile est bien intercepté par la MMU.

Q28 message "stack overflow"

Faites un peu de consmétique, en déplacant votre handler de faute de page, depuis `src/main.c` vers `src/mmu.c`. Modifiez `include/mmu.h` en conséquence. En effet, il devient de plus en plus clair, que la programmation de cette intérruption est intimement liée à la configuration choisie pour la mmu.

Puis implémentez (dans mmu.c) un fonction `void *get_cr2()` qui retourne l'adresse à cause de laquelle la faute à eu lieu.

Et enfin modifiez votre handler d'interruption pour que lorsqu'une faute de page se produit, le message indique "stack overflow" plutot que "page fault" lorsque l'erreur vient d'un dépassement de pile par le programme utilisateur. Biensur dans les autres cas, le message générique reste pertinent.

Q29 analyse

Expliquez comment il serait possible de détecter un débordement de pile sans avoir recours à cette utilisation de la mmu. Puis expliquez quel sont selon vous les avantages et les inconvéniants de l'utilisation de la mmu pour empêcher les débordements de pile d'exécution.

5.2 Extension dynamique de pile

La principale motivation de l'introduction de la MMU dans les matériel est de faciliter la gestion de la mémoire physique. En ce qui concerne la pile d'exécution d'un programme, comme vous pouvez l'imaginer, certains programmes utilisent de la récursivité et ont donc besoin d'une pile d'appel importante. Au contraire, d'autres programmes, plus procéduraux, n'ont besoin que d'une pile d'appel trés modeste.

Dans cette section vous allez allouer des pages de mémoire, pour la pile d'exécution, au fur et à mesure des besoins de l'application. L'idée est la suivante. Le noyau va positionner la pile du programme *user* sur une page de mémoire virtuelle non présente. Lorsque l'application va lire ou écrire dans la pile, une faute de page va donc être produite. Le système d'exploitation va alors pouvoir allouer une page de mémoire, l'associer à la pile du programme exécuté et reprendre son exécution.

Ainsi la mémoire est donnée à l'application au fur et à mesure des besoins et non *a priori*. Cependant, en un sens, le système d'exploitation ment à l'application. Il prétend lui donner de la mémoire sans le faire vraiment.

Pour cela vous allez implémenter une fonction d'allocation et de libération de page mémoire. Ensuite, vous modifierez le handler de faute de page pour qu'il alloue de nouvelles pages, au fur et à mesure des besoin. Si la limite maximale et atteinte, l'erreur "Stack overflow" sera affichée.

Puisque l'allocation se fera au besoin, il n'est plus utile de déclarer une variable `user_stack`. La pile sera initialement placé à la fin de la `user_page_table`. La dernière entrée ne sera pas associée une page, et au premier `push` du module `src/user.o` une faute déclanchera l'allocation d'une première page de pile d'exécution.

Q30 Structure d'allocation

Pour pouvoir allouer des pages, vous allez avoir besoin de savoir quelles pages de mémoire physique sont libres et quelles pages sont occupées. Pour cela vous allez définir une structure de donnée qui associe à chaque page (des 16 premiers Mo) un entier. Cet entier vaudra -1 si la page est reservée. sinon, il contiendra le numero d'une page suivante libre. Par ailleur, une variable globale `first_free_page` donnera le numero de la première page de mémoire physique libre. Déclarez ces structures de données dans le fichier `src/mmu.c`

Q31 initialisation

Cette structure d'allocation doit être correctement initialisée pour pouvoir fonctionner. Toutes les pages associées au noyau, ainsi que toutes les pages associées au programme `src/user.o` sont en quelque sorte, déjà allouées. La structure de donnée doit donc être initialisées en conséquence. La première page libre est donc celle qui suit la dernière page du module `src/user.o`. La page libre suivante, est indiquée dans la table à cet index, et ainsi dessuite jusqu'à la dernière page qui indique `-1` comme page suivante, puisqu'il n'y en a plus après elle. **N.B.** Comme la page `0` peut potentiellement être libre, vous éviterez d'utiliser `0` comme numero de page nulle. la valeur `-1` lui est préférable. Modifiez la fonction `void setup_mmu()` en conséquence.

Q32 `void *alloc_page()`

L'implémentation de `alloc_page()` est maintenant triviale. Il suffit de retourner la première page libre, après avoir indiquer que la future première page libre sera la suivante de la première page libre actuelle. Dans le tableau des pages à l'indice retourné, il doit y avoir la valeur `-1` qui signifie : page allouée. Comme la fonction alloc_page retourne l'adresse d'une page et non un numéro de page vous pourrez utiliser votre fonction `void *addr_for_page(int page)` pour faire la conversion sur la valeur retournée. Implémentez la fonction `void *alloc_page()`.

Q33 `void free_page(void *)`

Il faut maintenant pouvoir libérer les pages allouées. Il s'agit simplement d'insérer en tête de liste la page à libérer, et d'indiquer que la page libre qui la suit est l'ancienne tête de liste. Implémentez la fonction `free_page()`.

Q34 Pile d'appel virtuelle

Calculez les valeurs de idx1 et idx2 pour que la pile d'appel soit à la dernière page de mémoire virtuelle associée à la table `user_page_table`. En considérant que l'instruction `push` décrémente le sommet de pile **avant** d'écrire la valeur. Quel doit être l'adresse de la pile pour que l'instruction écrive sur cette page ?

Q35 Implémentation

Modifiez le handler de faute de page, pour qu'il alloue une page à chaque fois qu'un accés à la pile échoue et qu'il retourne à l'exécution plutot que d'échouer. Ainsi le programme sans appel récursif infini doit pouvoir s'exécuter normalement. Fixez une limite de 1Mo à l'extension de la pile. Laissez

grandir la pile jusqu'à cette valeur de seuil et afficher "stack overflow" lorsque le seuil est atteint. vérifiez le bon fonctionnement dans ce cas, en testant l'extension de la pile jusqu'au "stack overflow".

/ ! \ Félicitation vous avez implémentez votre premier mécanisme de virtualisation !

---

6 Processus lourd

Vous allez maintenant implémenter la notion de processus lourd. Un processus lourd est un contexte d'exécution associé à un espace de mémoire virtuelle qui lui est propre. Chaque programme en cours d'exécution s'exécute dans « sa » mémoire, c'est-à-dire, qu'il lit et écrit dans des pages qui lui sont propre, distinctes de celles des autres programme. En plus de sa pile d'exécution, chaque processus a donc un plan d'adressage distinct, configurer par le registre `%cr3`. Changer de processus, c'est changer d'espace d'adressage virtuel. Chaque processus à donc une `directory_table` qui lui est propre.

L'API POSIX que vous connaissez, définit la fonction `fork()` pour créer des processus. Dans cette API chaque processus est identifié par un numéro unique appelé `pid`. C'est donc cette fonction que vous allez implémenter maintenant.

Pour mémoire, le principe de la fonction `fork()` est de dupliquer la mémoire d'un processus pour en créer un second. Le nouveau processus ainsi créé est donc une copie parfaite du précédent. La seule différence entre les deux et la valeur retournée par la fonction `fork()`. L'original reçoit en réponse de l'appel le `pid` du processus créé, alors que le nouveau processus qui poursuit son exécution par le même retour de fonction `fork()` reçoit lui, comme valeur : 0. Cette différence permet à l'orignal et à la copie d'adopter, par la suite, un comportement différent.

Si la mémoire du processus initial est dupliquée, il n'y a aucune nécessitée de dupliquer les pages du noyau lui-même. En effet, quelque soit le programme utilisateur exécuté, le système d'exploitation reste unique.

Pour implémenter la duplication d'un processus, il faut aussi conserver l'état d'avancement de l'exécution de chaque processus. Cet état d'avancement est dans la pile d'exécution qui a été dupliquée, encore faut-il connaitre :

1. la plan d'adressage associé au processus (tel qu'il est donné au registre `%cr3`) et
2. le sommet de la pile (définit par les registres `%esp` et `%ebp`) ainsi que
3. le `pid` du processus en train de s'exécuter.

Cela constitue un contexte d'exécution.

Une fois un contexte d'exécution associé à chaque processus par le noyau, il devient possible d'implémenter la fonction système `yield()` qui passe d'un processus au suivant. Ce faisant elle change le plan d'adressage virtuel. C'est précisément parce que le plan d'adressage change d'un processus à l'autre que l'on parle de processus lourd. En effet, Cette reconfiguration de la mémoire virtuelle à chaque quantum de temps à un effet non négligeable sur les performances du programme. Les processus légers, qui partagent la même mémoire virtuelle sont plus efficaces. Mais ils ne sont pas isolés les uns des autres, et un bug dans l'un peut entrainer un plantage de tous.

Une fois votre premier programme `src/user.c` correctement démarrer, de tel sorte qu'il soit attaché à un contexte d'exécution et ordonnancé par la fonction système `yield`, il devient possible d'implémenter la fonction `fork` qui créée une copie de ce premier processus et l'intègre dans l'ordonnancement des programmes.

Pour réaliser la fonction `fork` vous allez donc :

1. copier la mémoire virtuelle d'un processus ;
2. gérer l'activation du premier processus ;
3. implémenter le changement d'un processus à un autre ;
4. réaliser le clonage à chaud de l'état d'un processus ;
5. gérer la terminaison d'un processus en libérant la mémoire associée.

6.1 Copie mémoire

Pour implémenter le clonage vous allez réaliser la fonction :

```
struct Page_Directory_s *clone_page_directory(struct Page_Directory_s
 *src_pd);
```

Cette fonction alloue et initialise une nouvelle `page_directory`. Comme pour l'original, toutes les entrées sont « non présentes » à l'exception des deux premières. La première est simplement la même de celle d'origine puisqu'elle concerne le code noyau, qui est unique. La seconde référence une nouvelle `page_table` au sein de laquelle chaque entrée est une copie de la source, mais, lorsqu'une entrée est présente, elle référence une nouvelle page, qui est une copie de l'originale.

Q36 `void page_copy(...)`

Il s'agit en premier lieu de disposer d'une fonction permettant de copie une page de mémoire. Cette fonction de prototype `void page_copy(char *pg_src, char *pg_dst)` copie la page de mémoire pointée par `pg_src` dans la page de mémoire `pg_dst`. Comme ces adresses désigne de la mémoire physique, il convient d'arrêter la même virtuelle avant de procédé à la copie, puis de la

réactiver ensuite. Notez que pour réaliser la copie en elle-même, vous réaliserez une simple boucle, la fonction `memcpy` dépendant de la `libc` n'est évidement pas utilisable dans votre noyau.

Q37 `clone_page_directory(...)`

Implémentez la fonction `struct Page_Directory_s *clone_page_directory(struct Page_Directory_s *src_pd);`. Cette fonction alloue et initialise une nouvelle `struct Page_Directory_s` comme expliqué précédemment, ainsi qu'une nouvelle `user_page_table` avec la fonction `alloc_page()`. Ensuite des pages additionnelles sont allouées pour chaque entrée de la `user_page_table` valide. Vous pourrez ensuite utiliser la fonction `page_copy` pour cloner le contenu du processus.

Q38 `free_page_directory(...)`

Implémentez la fonction `void free_page_directory(struct Page_Directory_s *src_pd);`. Cette fonction libère toutes les pages *user* allouées dans le `page_directory` donné en paramêtre, ainsi que la `user_page_table` et `le page_directory` lui-même. Bien sur il ne s'agit pas de libérer les pages *kernel* (associées à l'entrée 0), qui elles sont communes à tous les processus.

6.2 Activation

En clonant le contenu de la mémoire virtuelle du programme utilisateur, vous avez cloné, *de facto* la pile d'appel du mode *user*. Celle que vous avez alloué dynamiquement. Ainsi vous avez créé un nouveau processus, clone du processus initial, mais avec une pile dont le contenu pourra évoluer différemment dans le future, sans écraser l'original.

Cependant, pour l'instant, vous ne vous êtes pas préoccupé de la pile d'appel noyau. Celle qui exécute votre fonction `main` au moment du *boot*. Cette pile est définit dans le *link script* de compilation, et elle est mise en place dans le fichier `src/0boot.s`. Pour les plus curieux d'entre vous, cette pile est définit comme pile de réception d'interruption dans le fichier `src/gdt.c`.

Q39 *kernel stack*

Le but est donc de créer une pile d'exécution noyau par programme exécuté. En l'état l'entrée 0 de chaque `page_directory` associée à un processus est systématiquement liée à la même `page_table` initiale. Vous allez don déplacer la pile noyau dans la mémoire associée à l'entrée 1 des

différents `page_directory`. Ce faisant, en changeant de processus, le microprocesseur changera de mémoire utilisée pour la pile *kernel*.

Modifiez le *link script* pour déplacer la `_kernel_stack` à la fin de la 4ème page (entrée 3) de la `user_page_table`. Déplacez le programme `src/user.o` juste après ces 4 premières pages. Modifiez la fonction `setup_mmu()` pour que les pages de la pile noyau ainsi déplacée ne soient accessibles qu'au noyau. Et vérifier que tout fonctionne correctement une fois cela fait.

Q40 Contextes d'exécution

Pour pouvoir passer différents processus, vous allez devoir gérer un ensemble de contexte. Un contexte d'exécution est, comme cela a été vu précédemment, un sommet de pile d'appel (géré par `%esp` et `%ebp` sur les processeurs intel) mais aussi un espace d'adressage virtuel propre (géré par le registre `%cr3`).

Dans le fichier `include/ctx.h` donnez l'implémentation d'une structure de donnée `struct ctx_s` qui sera utiliser pour stocker les informations relatives à un processus. Ces informations sont :

- le `pid` (numéro unique, associé à chaque processus, et égal à 1 pour le premier processus) ;
- le `cr3` qui définit le plan d'adressage du processus ;
- le `esp` et `ebp` qui définissent la position de reprise de la pile d'exécution ;
- et enfin `next` qui donne l'adresse du processus suivant dans la liste.

La variable globale `current_ctx` déclarée dans `src/ctx.c` donne l'adresse du contexte courant.

Q41 allocateur de `struct ctx_s`

Au besoin, le noyau alloue une nouvelle structure `ctx_s` en mémoire. Cette structure sera libérée lorsque le processus dont elle stocke l'état terminera.

1. Fixez une constante `MAX_CTX` dans `include/ctx.h` qui définit le nombre maximum de contexte simultanément disponible.
2. Déclarez dans `src/ctx.c` un tableau `ctxs` de `MAX_CTX` contextes en variable globale, ainsi qu'une variable globale `first_free_ctx`.
3. Implémentez une fonction `void init_ctx()`. Cette fonction initialise la variable `first_free_ctx` avec l'adresse de la première entrée du tableau. Le champs `next` de la première entrée donne l'adresse de la seconde entrée, et ainsi de suite jusqu'à la dernière entrée du tableau qui donne `NULL`. Ainsi vous avez une liste de `struct cts_s` libres.
4. Implémentez une fonction `struct ctx_s *alloc_ctx()` qui retourne l'adresse d'une `struct cts_s` libre. Pour cela elle retourne simplement la valeur stockée dans

`first_free_ctx` et modifie cette variable globale pour qu'elle pointe sur le prochain (`next`) *free_ctx*. Bien sur, si `first_free_ctx` est `NULL` la fonction retourne `NULL` (il n'y a plus de `struct ctx_s` disponible dans votre noyau).

5. Implémentez une fonction **`void`** `free_ctx(struct ctx_s*)` qui insère en tête de la liste des `struct ctx_s` libres la `struct` passée en paramètre.

Assurez-vous encore que la fonction `alloc_ctx` retourne une structure avec un champs `pid` correctement initialisé (incrémenté de 1 par rapport au dernier contexte retourné).

Q42 Un premier `ctx_s`

Modifiez votre fonction de boot (**`void`** `main()`) pour qu'elle initialise votre gestion des contextes (grace à `init_ctx()`). Puis qu'elle alloue un premier contexte d'exécution, qu'elle l'associe à `current_ctx` pour le premier processus créé, avant de le lancer (le processus dans src/user.c"). Le suivant de ce premier processus actif sera lui-même puisqu'il est le seul actif pour l'instant.

6.3 Changement de processus

Q43 *syscall* `getpid`

Implémentez maintenant un nouvel appel système : `getpid()`. Le fonction noyau `getpid()` retourne le pid du processus courant. Pour cela, la fonction en *user land* déclare une variable locale, et donne l'adresse de cette variable dans `%edx`. La fonction `sys_call_handler` du système récupère l'adresse de la variable locale écrite dans le registre `%edx` passé en paramêtre, et écrit la valeur attendue à cette adresse. Enfin la fonction *user land* peut venir lire le resultat pour le retourner à l'appelant.

Testez le bon fonctionnement de cette fonction.

Q44 `switch_to` v1.0

Vous allez maintenant implémenter une fonction **`void`** `switch_to(struct ctx_s *ctx)` qui sauvegarde le contexte d'exécution courant dans `current_ctx` avant de restaurer le contexte `ctx` qui devient le nouveau contexte courant.

La fonction `switch_to` assure la « commutation » d'un contexte à l'autre simplement en : 1. sauvegardant les registres de pile (`%esp` et `%ebp`) puis ; 2. affectant `ctx` comme nouveau `current_ctx` ; 3. restaurant les registres de pile du nouveau `current_ctx`, mais aussi (et surtout) le registre `%cr3` qui reconfigure la mémoire virtuelle pour le nouveau contexte d'exécution.

Implémentez la fonction **`void`** `switch_to(struct ctx_s *ctx)`.

Q45 *syscall* `yield`

Implémentez une fonction `kyield()` qui transfère simplement le contexte d'exécution d'un programme au suivant. Comme vous avez créé un anneau à partir de `current_ctx` l'implémentation de `yield` est triviale.

6.4 clonage à chaud

Q46 `switch_to` v2.0

En l'état, la fonction `switch_to` transfère le flot d'exécution vers un contexte donné en argument. Cependant, lors de la création d'un processus clone, il faut mettre le cpu dans la situation ou il pourra résumer un contexte d'exécution `switch_to` vers ce clone. Pour cela, vous allez faire évoluer la fonction `switch_to` pour qu'elle crée un clone, à partir du contexte courant, si NULL lui est passé en argument. Le clone est simplement inséré dans l'anneau à la suite du contexte courant. Comme le clone est une stricte copie du contexte courant, le sommet de pile (les registres `%esp` et `%ebp`) peut être sauvé dans le contexte courant. Cependant, le registre de configuration de la mémoire virtuelle, `%cr3` est celui obtenu par clonage (retournée par votre fonction `struct Directory_Entry_s *clone_page_directory(struct Directory_Entry_s *original);` et non l'original. Notez encore que lorsque `switch_to` est appelé avec `NULL` il ne change pas le contexte courant, et ne transfère donc pas l'exécution vers le clone. La « commutation » aura lieu au prochain appel de `kyield()` puisque ne nouveau processus est inséré « après » le processus courant.

Q47 *syscall* `fork()`

Implémentez un nouvel appel système `void fork()`. Cet appel système appelle la fonction `kfork()` qui appelle à son tour `switch_to()` avec en paramètre `NULL` pour créer un processus clone.

Q48 retour de `fork()`

Selon le standard posix, la fonction système `fork()` retourne 0 pour le processus fils, et le `pid` du du processus fils, pour le processus père. Implémentez ces valeurs de retour, et vérifiez qu'elles sont correctes du point de vue du processus `src/user.c`.

6.5 Terminaison

Q49 *syscall* `exit`

Implémentez l'appel système `void exit()` qui termine un processus. Pour cela, l'appel retire le processus courant de la liste des processus éligible, en faisant pointer son prédécesseur vers son suivant. Puis il libère le contexte (`struct ctx_s` associée). Enfin il libère l'ensemble des pages allouées à ce processus en appelant la fonction : `void free_page_directory(struct Page_Directory_s *src_pd);`.

Q50 user `_entry_point`

Lorsque la fonction `void user_main()` termine, il faut que le processus ce termine', en appelant la fonction système `void exit()`. Pour cela, plutôt que d'appeler la fonction implémentée par le developpeur, le noyau va appeler la fonction `void _entry_point()` (de l'espace *user*). Cette fonction appelle la fonction `void user_main()` et si `user_main` termine, `_entry_point` appelle `exit`.

Implémentez la fonction '`void _entry_point()` et assurez-vous que ce soit elle qui soit appelée par le noyau.

6.6 Validation

Q51 Sécurité

Expliquez pourquoi il est « matériellement » impossible au processus fils de consulter une information secrète rangée dans une variable du père, **après** la création du fils. Est pourquoi il le peut, avant ? Répondez dans `rendu.md`mais proposez un programme `src/user.c` qui illustre votre propos.

6.7 Clonage virtuel

7 ... To Be Continued ...